

Proof-Carrying Pipelines: Offloading CI/CD Gate Execution to Untrusted-but-Identified Machines with Cryptographic Attestations

Jack Andrés Cid *Independent work — v1.0, August 2026*

Abstract

Continuous integration pipelines re-execute the same validation gates — linters, static analyzers, compilation, unit tests, contract checks — on shared cloud runners for every push, even when the developer already ran exactly those gates locally seconds earlier. This redundancy is treated as the price of trust: the pipeline cannot know that the local run happened, ran the *current* rules, or ran them against the *pushed* content. We present **Proof-Carrying Pipelines (PCP)**, a pattern in which pipeline gates execute on untrusted-but-identified machines (typically developer workstations) and their verdict travels with the commit as a cryptographic attestation that binds together (i) the exact content hash, (ii) the digests of the validation tools, (iii) the version of the rule set, and (iv) a timestamp, signed by an organization-held key (HSM/KMS) that the machine can invoke but never possess. A lightweight **attest-and-skip gate** at the pipeline choke point verifies the attestation in seconds and elides redundant re-execution; any missing, stale, drifted, or invalid proof triggers the full pipeline as a fail-closed fallback. A freshness-and-drift lock keeps local tooling pinned to the pipeline's sources of truth, so a proof from an outdated toolchain is never accepted. PCP occupies a deliberately pragmatic point in the design space: it provides *organizational* integrity — attributable, revocable, audited execution by enrolled identities — without requiring trusted execution environments or fully reproducible builds, and degrades gracefully to classic CI when its assumptions fail. We describe the protocol, its threat model, a reference implementation, and a production-shaped case study on an enterprise data platform where contract, SQL-model and DAG validation gates were offloaded with first-push compliance preserved. The name is a deliberate homage to proof-carrying code: the artifact carries the evidence; the checker stays cheap.

1. Introduction

Two forces are colliding in modern software delivery. Pipelines keep absorbing more validation — style, static analysis, policy-as-code, schema and contract checks, unit tests — because centralizing them is how organizations guarantee they run. Meanwhile developer machines have become absurdly capable and sit idle during every CI wait. The result is a familiar loop: run the checks locally, push, then wait for a shared runner to run the same checks again, queued behind every other team, at cloud prices.

The waiting is not the deep problem; *trust* is. The pipeline re-executes because it cannot believe three things about a local run: that it **happened at all**, that it ran over **exactly the content that was pushed**, and that it used the **currently approved tools and rules** rather than last month's. Any mechanism that establishes those three facts more cheaply than re-execution converts redundant cloud compute into a signature check.

Culturally, the demand exists. Basecamp's `gh-signoff` (2025) lets a developer mark a commit green after running checks locally — by plain self-declaration, with no binding to content, tools, or rules. Its reception captured both halves of the situation: enthusiasm for reclaiming local execution, and immediate criticism that nothing prevents a stale toolchain, a dirty approximation of the pipeline, or a simple lie. PCP is, in one sentence, `gh-signoff` with the missing cryptography and the missing operational discipline.

Contributions.

1. A named, general pattern — *Proof-Carrying Pipelines* — for offloading pipeline gate execution to untrusted-but-identified machines under organizational key custody, with verify-and-skip semantics at the pipeline.
2. A minimal attestation binding — content hash $\times$ tool digests $\times$ ruleset version $\times$ identity $\times$ time — that is necessary and, we argue, sufficient for the elision decision.
3. Two protocol obligations that prior self-attestation approaches lack: a **fail-closed fallback** (absent/invalid proof $\Rightarrow$ the full pipeline runs, so PCP can never be *less* safe than classic CI) and a **freshness-and-drift lock** (proofs from drifted tooling are rejected and the local bundle refuses to sign until it self-updates).
4. A reference implementation and a case-study instantiation on an enterprise data platform.

2. Related work

Supply-chain attestation. in-toto provides signed link metadata proving that supply-chain steps were performed by authorized functionaries, verified against a layout at the end of the chain. PCP borrows its core insight — sign the *execution of a step*, not just the artifact — but repurposes it for a different decision at a different point: elide *re-execution inside the pipeline itself*, with an explicit fallback path and an economics-driven framing (compute offload). SLSA provenance, sigstore/cosign and GitHub Artifact Attestations sign what was built and where; they document provenance rather than replace pipeline work.

Hardware-rooted approaches. Attestable Builds (2025) and the evidence-driven protocol of Castillo et al. (2026) obtain stronger guarantees — attested execution state — by running build/test steps inside TEEs (e.g., Intel TDX), often combined with reproducible builds and transparency logs. These designs remove the need to trust the executing party at all. PCP is the pragmatic complement: no TEE requirement, no reproducibility requirement, deployable today on unmodified laptops, at the cost of trusting enrolled identities within an audited, revocable, fail-closed envelope. Where TEEs are available, they slot into PCP as a stronger signer.

Determinism-based trust. Tweag's untrusted-CI work and Trustix trust build *outputs* by content-addressing and cross-builder comparison under Nix. This verifies results without signatures but requires deterministic builds and addresses artifact caching rather than policy gates (many of which — linters, rule engines — are cheap to re-run but *matter* because their rule sets change).

Build caches. Bazel remote caches, Nx Cloud and Turborepo skip recomputation keyed by input hashes. The known weakness is trust in cache writers ("can you trust your build cache?"): access to the cache is authority to inject results. Signature support where it exists covers transport integrity of artifacts, not "the gates ran, with these tools, under these rules." PCP is the missing trust layer for the *skip* decision, applied to pipeline gates.

Self-attestation. gh-signoff demonstrates the demand and the gap simultaneously: local sign-off with no cryptographic binding. PCP's attestation is precisely the set of facts whose absence its critics pointed at.

Proof-carrying code. Nacula and Lee's PCC (1996) established the asymmetry PCP exploits: the untrusted producer does the expensive work and ships evidence; the consumer runs a cheap checker. PCP's "proofs" are attestations of gate execution rather than formal proofs of program properties — the checker verifies *who ran what over what content with which tools*, not the semantic property itself. The homage is intentional and the limitation explicit (§7).

3. Threat model

Assets. Integrity of the deployment gate decision: nothing reaches protected branches / deployment without the required gates having passed over the exact delivered content.

Adversaries considered. (A1) A *careless* developer: stale tools, dirty tree, forgot to run checks. (A2) A *pressured* developer attempting to shortcut gates without deliberately attacking cryptography. (A3) An *external* attacker who compromises the transport, the repo remote, or replays old proofs. (A4) *Drift*: the organization changes rules/tools and outdated local bundles keep signing.

Out of scope (declared, not hidden). (A5) A developer with valid signing rights who deliberately fabricates a verdict by tampering with the *local execution itself* (e.g., patching the validator binary before it runs) is not prevented by PCP without hardware attestation — this is the TEE boundary. PCP mitigates A5 by pinning tool *digests* (the attestation names the exact container images/binaries), by audit logging every signature (KMS/Cloud Audit), by supporting random re-verification (the pipeline re-runs a sampled fraction of skipped gates), and by making the skip *revocable per identity*. Organizations for which A5 is intolerable should combine PCP with TEE-backed signers or forgo elision.

Key custody. Signing keys live in an HSM-backed KMS; machines hold *invocation rights* (IAM), never key material. Compromise of a laptop yields the ability to sign — visible in audit logs, revocable in minutes — not the key.

4. The pattern

Roles. *Producer*: the identified machine executing gates (developer workstation, edge runner). *Signer*: org-held KMS key the producer may invoke. *Verifier*: the attest-and-skip gate in the pipeline. *Sources of truth*: the repositories/registries that define gates, tool images, and rule sets.

Protocol.

1. **Pin.** The local bundle materializes the gates from the sources of truth and records a lock: {source SHAs, tool image digests, ruleset digest, locked_at}.
2. **Execute.** Gates run locally — ideally in the *same container images* the pipeline uses.
3. **Bind.** On PASS, the producer canonicalizes a payload: {content: git_tree_hash, tools: [image digests], rules: ruleset_digest, verdict, identity, timestamp}.
4. **Sign.** The payload digest is signed via KMS (e.g., EC-P256/SHA-256). The attestation travels with the commit (git note, trailer, or sidecar).

5. **Verify (attest-and-skip gate).** The pipeline recomputes the tree hash, fetches the public key, verifies the signature, and checks four predicates: $\text{signer} \in \text{enrolled set}$; $\text{tool digests} \in \text{currently approved set}$; $\text{ruleset digest} = \text{current}$; $\text{age} \leq \text{TTL}$.
6. **Skip or fall back.** All predicates hold $\Rightarrow$ redundant gates are elided (registration, deployment and any *non-redundant* stages still run). Any predicate fails $\Rightarrow$ the full pipeline executes. **Absence of proof is never an error; it is the classic path.**
7. **Drift lock.** The local bundle checks its lock against the sources of truth before signing; on drift or staleness it still validates but refuses to sign (advisory verdict) until self-updated. This makes rule rollout instantaneous: bumping the ruleset digest invalidates every outstanding proof at the verifier without coordinating with producers.

Design notes. The binding must be to the *tree hash* (content), not the commit hash, so history rewrites that preserve content preserve proofs, and any content change breaks them. TTL bounds replay of "honest but old" proofs (A3/A4). Random re-verification converts A5 from undetectable to statistically detectable. The verifier must be *cheap and boring* — one signature check and four set-membership/equality tests — because its cost bounds the pattern's benefit.

5. Reference implementation

The reference implementation (this repository) is ~500 lines of dependency-light Python + Bash: `pcp attest` runs declared gates (`gates.yaml`: command + pinned image digest each) in their containers, canonicalizes the payload (RFC 8785-style JSON), and signs via pluggable backends (local Ed25519 for the demo; Google Cloud KMS asymmetric-sign in production); `pcp verify` implements the attest-and-skip predicate set and emits an exit code suitable for `allow_failure: false` CI jobs; a `versions.lock` + `pcp update` pair implements the drift lock. A 30-line GitHub Actions / GitLab CI snippet shows the gate wiring with fallback.

6. Case study (anonymized)

On a large retail enterprise's self-service data platform, every push re-ran four gate families on shared runners: SQL-model rule validation (18 org rules), DAG static analysis, contract structure validation, and dbt compile+unit tests. We instantiated PCP with the platform's own validator containers and deployed rule documents as sources of truth. The local bundle reproduced all five CI gates bit-for-bit against a generated pipeline (same pinned dbt-core, same rule JSONs), reached PASS parity with the cloud pipeline on first push, and the drift lock correctly downgraded verdicts to advisory when the lock aged past 14 days or upstream SHAs moved. Wall-clock effect: the redundant portion of the pipeline (minutes of container startup, dependency resolution and re-validation on shared runners) collapses to a signature verification (seconds); compute shifts to machines already paid for and idle. The fail-closed property held trivially: proofs absent $\Rightarrow$ the untouched classic pipeline ran.

7. Security analysis and limitations

PCP's guarantee is *organizational*, not *physical*: "these gates passed over exactly this content, with exactly the approved tools and rules, executed under an enrolled, audited, revocable identity, recently" — it is not

"no party could have cheated." The elision decision inherits the strength of (a) key custody (HSM-grade), (b) identity enrollment (IAM-grade), (c) tool pinning (registry-grade), and (d) the audit/re-verification regime. A5 (malicious enrolled producer) is bounded, not eliminated: pinned digests force the attacker to tamper *around* the tools rather than with them; sampling re-verification bounds the expected survival of fabricated verdicts; audit logs make every signature attributable. Gates whose outcomes depend on environment (integration tests against live services) are poor PCP candidates and should stay in the pipeline; PCP targets *hermetic* gates (lint, static analysis, policy checks, compilation, unit tests), which in practice dominate redundant CI time. Finally, PCP does not accelerate cold-start consumers (a fresh clone still runs everything); its economics target the high-frequency inner loop.

8. Future work

TEE-backed signers as a drop-in producer upgrade; transparency-log countersigning of attestations (sigstore/Rekor) for third-party auditability; proof aggregation for monorepos (one attestation per affected target set); formalizing the verifier predicates; and measuring fleet-level compute displacement at scale.

References

- Necula, G. C., Lee, P. *Safe Kernel Extensions Without Run-Time Checking* / Necula, G. C. *Proof-Carrying Code*, POPL 1997.
- Torres-Arias, S. et al. *in-toto: Providing farm-to-table guarantees for bits and bytes*, USENIX Security 2019.
- SLSA — Supply-chain Levels for Software Artifacts. <https://slsa.dev>
- Sigstore. <https://sigstore.dev> · GitHub Artifact Attestations, 2024.
- *Attestable builds: compiling verifiable binaries on untrusted systems using trusted execution environments*, arXiv:2505.02521, 2025.
- Castillo, F., Brito, E., Pullonen-Raudvere, P., Werner, S., Tai, S. *An Evidence-driven Protocol for Trustworthy CI Pipelines*, arXiv:2605.21089, 2026.
- Tweag. *Untrusted CI: automatic trusted caching of untrusted builds with Nix*, 2019. · Trustix, nix-community.
- Nx. *Can You Trust Your Build Cache?* (blog). · Turborepo remote caching + artifact signature verification (docs).
- Basecamp. *gh-signoff* — <https://github.com/basecamp/gh-signoff>, 2025.